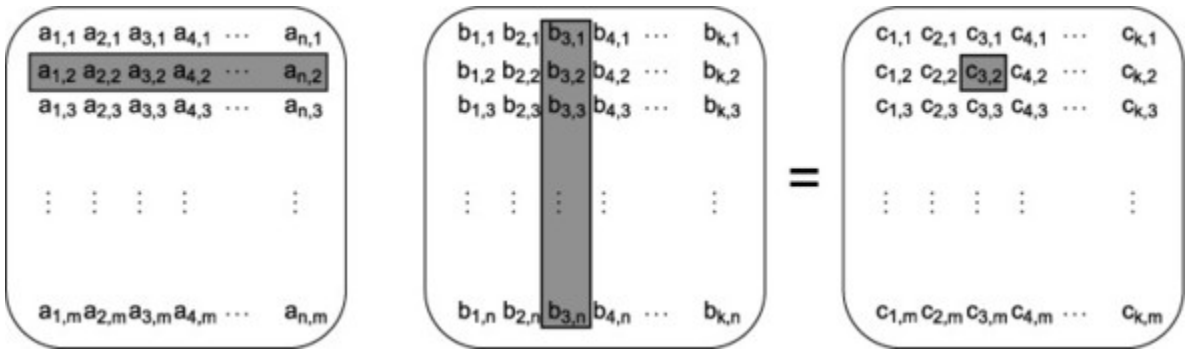
of the first *column* of the second matrix and add up the products to give the top-left element of the result. You then repeat this with the second row and the first column to give the second element in the first column of the result, and with the first row and second column to give the first element in the second column of the result, and so forth. This is shown in figure 8.3; the highlighting shows that the second row of the first matrix is paired with the third column of the second matrix to give the entry in the second row of the third column of the result.

Now let's assume that these are *large* matrices with several thousand rows and columns, in order to make it worthwhile using multiple threads to optimize the multiplication. Typically, a non-sparse matrix is represented by a big array in memory, with all the elements of the first row followed by all the elements of the second row, and so forth. To multiply your matrices you thus have three of these huge arrays. In order to get optimal performance, you need to pay careful attention to the data access patterns, particularly the writes to the third array.

There are many ways you can divide the work between threads. Assuming you have more rows/columns than available processors, you could have each thread calculate the values for a number of columns in the result matrix, or have each thread calculate the results for a number of rows, or even have each thread calculate the results for a rectangular subset of the matrix.

Figure 8.3. Matrix multiplication



Back in sections 8.2.3 and 8.2.4, you saw that it's better to access contiguous elements from an array rather than values all over the place,